SEMANA 14 · INTELIGENCIA ARTIFICIAL EN LA NUBE I

IA en la Nube I: Modelos, Tokens e Inferencia

Parte 1 de 4 · Modelos Fundacionales y Arquitectura de Inferencia

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 14 · Periodo 2026-2 · Barranquilla, Colombia

Parte 1 de 4 · Modelos Fundacionales y Arquitectura de Inferencia

Del Software Determinista al Probabilístico

De árboles de decisión con reglas fijas a espacios vectoriales continuos y probabilidades

Software Determinista

Lógica Booleana Explícita
Sentencias IF / ELSE fijas

Salida Idéntica Garantizada
Cero variabilidad de respuesta

Cómputo en CPU y RAM
Escalamiento horizontal tradicional

Modelos Probabilísticos (IA)

Pesos Sinápticos Matriciales
Miles de millones de parámetros

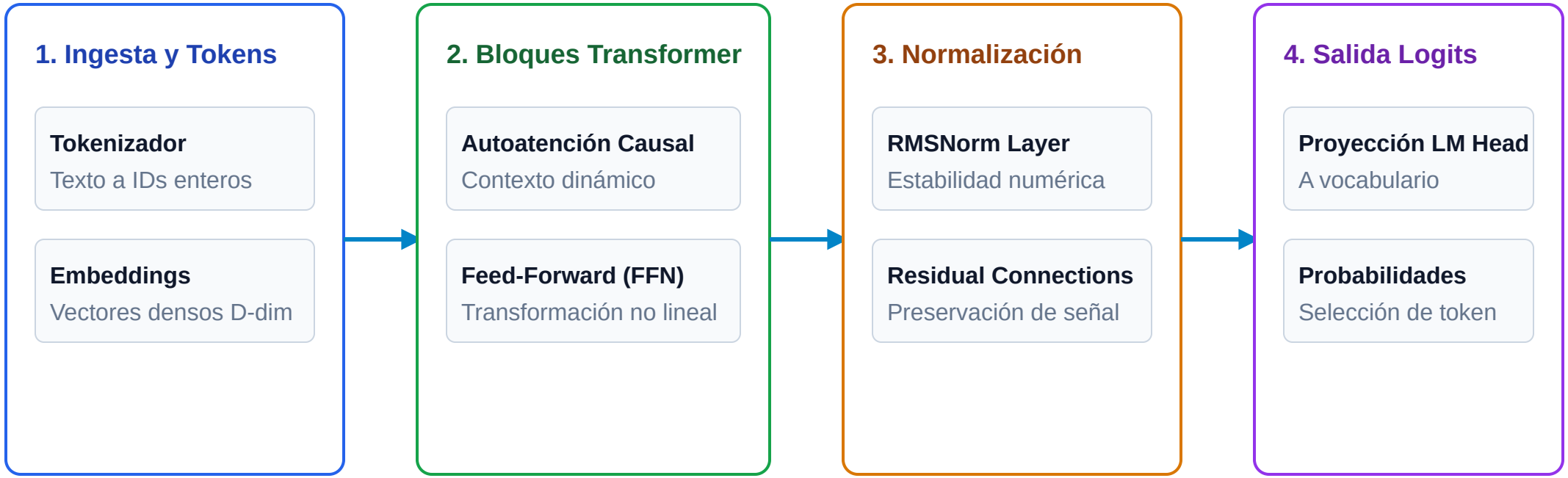
Distribución de Probabilidad
Generación estocástica contextual

Aceleración en GPU y HBM
Demanda masiva de ancho de banda

Fuente: AWS Machine Learning Lens · Well-Architected Framework

Anatomía de un Modelo de Lenguaje (LLM)

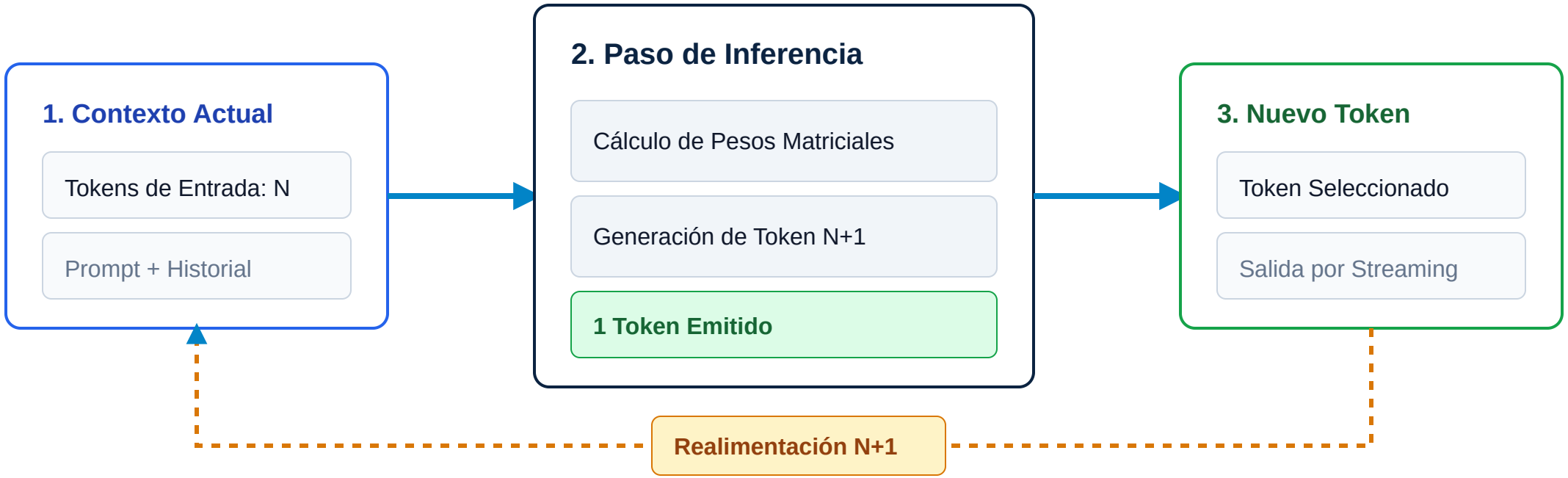
Capas constitutivas desde la entrada textual hasta la proyección de logits en memoria



Fuente: Vaswani et al. (2017) · Touvron et al. (2023) Llama Architecture

El Decodificador Causal y la Generación Token a Token

Mecanismo autorregresivo: cada nueva predicción se realimenta como entrada



Fuente: Vaswani et al. (2017) · Radford et al. (2019) GPT Architecture

Autoatención: Ponderación de Relevancia Contextual

Cómo los tokens establecen relaciones mutuas y su impacto en la memoria GPU

1. Consulta (Query)

Qué busca el token
Vector de intención actual

Enfoque Semántico
Interroga a tokens previos

2. Clave (Key)

Qué ofrece cada token
Etiqueta de contenido léxico

Cálculo de Afinidad
Matriz de atención mutua

3. Valor (Value)

Información transmitida
Contenido representacional

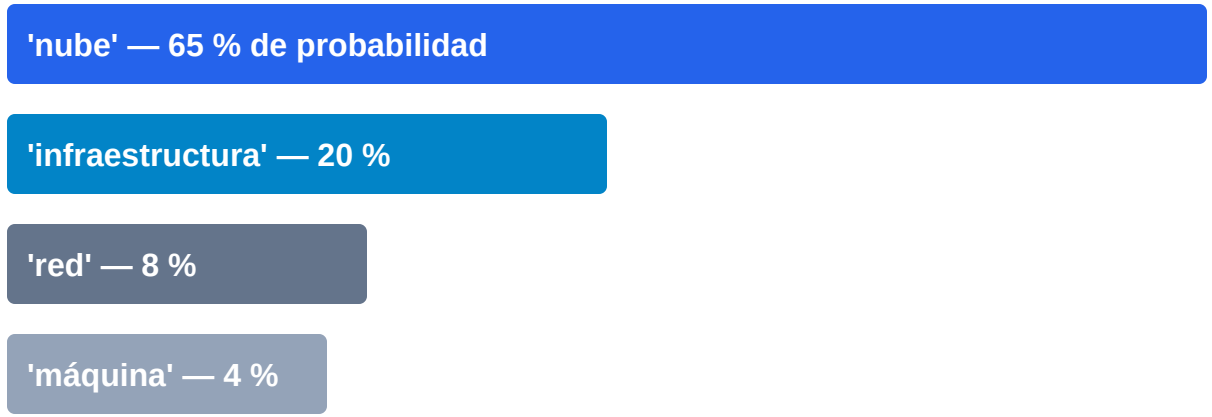
Costo de Memoria GPU
Crece con largo de contexto

Fuente: Vaswani et al. (2017) · AWS Bedrock Architecture Guide

Distribución de Probabilidad sobre el Vocabulario

De los logits numéricos a la asignación de probabilidades por token

Distribución de Tokens Candidatos para el Siguiente Paso



Resto del vocabulario (100.000+ tokens) se reparte el 3 % restante

Fuente: Holtzman et al. (2020) The Curious Case of Neural Text Degeneration

PARÁMETROS DE MUESTREO

Control de Entropía: El Impacto de la Temperatura

Ajuste de agudeza o dispersión en la distribución probabilística

T = 0.0 (Determinista)

Colapso en Token #1
Elige siempre el más probable

Uso: Código y SQL
Respuestas reproducibles

T = 0.7 (Balanceado)

Curva Natural Suave
Mantiene proporciones base

Uso: Chat Corporativo
Fluidez sin alucinaciones

T = 1.5 (Alta Entropía)

Distribución Aplanada
Tokens raros ganan peso

Riesgo: Alucinación
Pérdida de coherencia formal

Fuente: AWS Bedrock Inference Parameters Documentation

Estrategias de Recorte: Top-P y Top-K

Filtrado dinámico de la cola probabilística para eliminar salidas erráticas

Muestreo Top-K (Filtro Fijo)

Límite por Cantidad K
Conserva únicamente los K mejores tokens

Ejemplo: K = 50
Descarta el resto sin considerar probabilidad

Riesgo en Contextos Planos
Puede incluir tokens absurdos si K es alto

Muestreo Top-P (Núcleo Dinámico)

Masa de Probabilidad Acumulada
Corta cuando la suma alcanza el umbral P

Ejemplo: P = 0.90
Adapta el número de candidatos al contexto

Estándar en la Industria Cloud
Elimina colas largas con elegancia matemática

Fuente: Holtzman et al. (2020) Nucleus Sampling · AWS Bedrock Docs

Entrenamiento frente a Inferencia: Dos Mundos de Cómputo

Diferencias fundamentales en infraestructura, costo y requisitos de latencia

Entrenamiento (Fase de Creación)

Paso Adelante y Atrás (Backprop)
Ajuste de pesos mediante gradientes

Clústeres Masivos (Supercómputo)
Miles de GPUs interconectadas por InfiniBand

Inversión CapEx / Proyecto Finito
Ejecución batch orientada a throughput

Inferencia (Fase Productiva)

Solo Paso Adelante (Forward Pass)
Pesos estáticos de solo lectura

Servidores Cloud Escalables
1 a 8 GPUs por nodo o APIs Serverless

Costo Operativo OpEx Continuo
Servicio 24/7 con estricto SLA de latencia

Fuente: Patterson et al. (2021) Carbon Emissions and Large Neural Network Training

Inferencia en la Nube: Memoria frente a Cómputo

Por qué la generación autorregresiva está limitada por el ancho de banda de VRAM

Fase 1: Prefill (Prompt Inicial)

Procesamiento Paralelo
Todos los tokens del prompt a la vez

Limitado por Cómputo (Compute-Bound)
Alta saturación de núcleos Tensor

Métrica clave: TTFT (Time to First Token)

Fase 2: Decode (Generación Token)

Procesamiento Secuencial
Un token generado por cada ciclo

Limitado por Memoria (Memory-Bound)
Mover gigabytes de pesos para un token

Métrica clave: TPOT (Time Per Output Token)

Fuente: Dao et al. (2022) FlashAttention · NVIDIA H100 Architecture Whitepaper

Familias de Modelos: Propietarios frente a Abiertos

Comparativa arquitectónica de modelos gestionados y modelos de pesos abiertos

Modelos Gestionados (Bedrock)

Amazon Nova (Micro, Lite, Pro)
Modelos fundacionales nativos de AWS

Anthropic Claude (3.5 Sonnet / Haiku)
Alta capacidad de razonamiento y código

Consumo Serverless por Tokens
Cero gestión de servidores o GPUs

Modelos de Pesos Abiertos

Meta Llama 3 (8B, 70B)
Pesos descargables para autohospedaje

Mistral AI (NeMo, Large)
Arquitecturas eficientes y multilingües

Control Soberano en EC2 o Local
Ejecución privada sin enviar datos fuera

Fuente: AWS Bedrock Supported Models · Meta Llama Documentation

Regla Práctica: Parámetros y Demanda de Memoria VRAM

Fórmula matemática de dimensionamiento para alojar modelos en GPUs de la nube

VRAM Total = (Parámetros × Bytes/Precisión) + KV Cache + 20 % Margen

Modelo 8B (FP16: 2B/parám)

Pesos base: 16 GB

KV Cache (4k tokens): ~2 GB

Margen de activación: ~2 GB

Total: ~20 GB VRAM

Hardware: 1 GPU NVIDIA A10G (24GB)

Modelo 8B (INT4: 0.5B/parám)

Pesos cuantizados: 4 GB

KV Cache optimizado: ~1.5 GB

Margen de activación: ~1 GB

Total: ~6.5 GB VRAM

Hardware: 1 GPU Local o T4 (16GB)

Modelo 70B (FP16: 2B/parám)

Pesos base: 140 GB

KV Cache multi-usuario: ~15 GB

Margen operativo: ~15 GB

Total: ~170 GB VRAM

Hardware: 4 a 8 GPUs (ej. 4x A100 80GB)

Fuente: NVIDIA Technical Blog · Sizing Deep Learning Inference Systems

Caso Ilustrativo: Clasificación Documental

Caso ilustrativo de arquitectura, no es una empresa real



Fuente: AWS Solutions Architecture · Enterprise Document Processing Pattern

SÍNTESIS CONCEPTUAL

Cuatro Principios de la Inferencia Fundacional

Reglas de oro que gobiernan el comportamiento de LLMs en la nube

1. Naturaleza Autorregresiva Secuencial

La generación token a token impone latencia acumulada

Cada nuevo token requiere una pasada completa por la red

La transmisión streaming es obligatoria para la experiencia

2. El Cuello de Botella es la Memoria VRAM

La fase Decode está limitada por el ancho de banda HBM

Mover los gigabytes de pesos domina sobre el cálculo FLOPS

El dimensionamiento se guía por VRAM y concurrencia

3. Control Estocástico mediante Muestreo

Temperatura cero para código, SQL y tareas deterministas

Top-P adaptativo para podar colas de baja probabilidad

Prevención de alucinaciones desde la configuración del API

4. Arquitectura en Cascada FinOps

Combinación de modelos pequeños para el 90 % del volumen

Enrutamiento condicional hacia modelos de frontera

Optimización drástica de costos sin sacrificar calidad

Fuente: AWS Well-Architected Framework: Machine Learning Lens

HOJA DE RUTA

Hacia la Parte 2: El Universo de los Tokens

De los modelos conceptuales a la unidad fundamental de facturación y contexto

Contenidos de la Parte 2

1. Tokenización con BPE y Disparidad Lingüística
Por qué el idioma español consume más tokens y cómo optimizarlo

2. Ventanas de Contexto y Complejidad Cuadrática
El impacto en memoria del largo de secuencia y el KV Cache

3. Tarifas Oficiales Cloud y Prompt Caching
Costos reales en Amazon Bedrock y estrategias de reutilización

SEMANA 14 · INTELIGENCIA ARTIFICIAL EN LA NUBE I

IA en la Nube I: Modelos, Tokens e Inferencia

Parte 2 de 4 · Tokens, Ventanas de Contexto y Economía Cloud

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 14 · Periodo 2026-2 · Barranquilla, Colombia

Parte 2 de 4 · Tokens, Ventanas de Contexto y Economía Cloud

¿Qué es un Token? De Caracteres a Subpalabras

Segmentación estadística de texto en unidades numéricas computables

Por Caracteres

Secuencias Muy Largas
1 carácter = 1 paso en la red

Poco Significado Semántico
Satura la ventana de contexto

Por Palabras Completas

Vocabulario Inmanejable
Millones de entradas requeridas

Fallo con Palabras Raras
Tokens desconocidos [UNK]

Por Subpalabras (BPE)

Equilibrio Óptimo
Fragmenta raíces y sufijos

Cero Palabras Desconocidas
Estándar universal en LLMs

Fuente: Sennrich et al. (2016) Neural Machine Translation of Rare Words with Subword Units

Byte-Pair Encoding (BPE): Mecanismo de Fusión

Construcción iterativa de vocabulario agrupando pares de caracteres frecuentes

Paso 1: Caracteres Base

['c', 'l', 'o', 'u', 'd']

Conteo de pares adyacentes
Frecuencia en corpus

Paso 2: Fusión Iterativa

Par más común: 'o' + 'u'

Crea nuevo token: 'ou'
Actualiza diccionario base

Paso 3: Token Final

Token ID: 'cloud' = #8432

1 token representa la palabra
Máxima eficiencia computacional

Fuente: Gage (1994) A New Algorithm for Data Compression · Hugging Face Tokenizers

El Desafío del Idioma Español en la Tokenización

Por qué procesar español consume entre un 20 % y 40 % más tokens que inglés

Texto en Inglés (Alta Representación)

"Cloud infrastructure management"

Resultado: 3 Tokens
['Cloud', 'infrastructure', 'management']

Ratio Medio: ~1.0 token / palabra

Texto en Español (Mayor Fragmentación)

"Gestión de infraestructura en la nube"

Resultado: 7 a 9 Tokens
['Gest', 'ión', ' de', ' infra', 'estructura', ...]

Ratio Medio: ~1.3 a 1.4 tokens / palabra

Fuente: Petrov et al. (2023) The Language Barrier in Large Language Models

Vocabularios Modernos: De 32k a 200k Tokens

Ampliación de diccionarios para reducir fragmentación y mejorar soporte multilingüe

Vocabularios Antiguos (32.000 Tokens)

Generación Previa (Llama 2)
Optimizado casi exclusivamente para inglés

Severa Fragmentación en Español
Palabras comunes divididas en 3 o 4 partes

Menor consumo de memoria en capa de embedding

Vocabularios Modernos (128k a 200k)

Modelos Actuales (Llama 3 / Claude 3.5)
Vocabulario BPE de 128.000 a 200.000 tokens

Compresión Eficiente en Español
Reduce hasta un 25 % el número de tokens

Aumenta tamaño de tabla de embedding en VRAM

Fuente: Meta Llama 3 Technical Report (2024) · Anthropic Claude Specifications

La Ventana de Contexto (Context Window)

Capacidad máxima de tokens simultáneos procesados en memoria activa

Estructura Interna de la Ventana de Contexto

Instrucciones Base

System Prompt

Reglas y políticas

Documentación de Soporte (RAG)

Manuales, PDFs, políticas internas

Hasta cientos de miles de tokens

Interacción y Salida

Pregunta del usuario

Espacio para la respuesta

Capacidad Total de Inferencia: 128k a 200k Tokens (Bedrock Claude 3.5 / Nova)

Fuente: Anthropic Documentation · AWS Bedrock User Guide

Evolución de la Ventana de Contexto (2020–2026)

De los 2.048 tokens originales a millones de tokens en memoria continua

Año 2020
2.048 Tokens
GPT-3 estándar

Año 2023
8k - 32k Tokens
Llama 2 / GPT-4

Año 2024
128k - 200k
Claude 3.5 / Llama 3

Año 2026
1M - 2M+ Tokens
Gemini / Nova Long

Fuente: Anthropic, Meta & AWS Release Milestones (2020–2026)

BARRERA FÍSICA

Complejidad Cuadrática: La Consecuencia Arquitectónica

Por qué duplicar el contexto cuadruplica el costo de memoria y latencia

Contexto Corto (4.000 Tok)

Operaciones Matriz
16 millones de cálculos

Demanda de Memoria
Bajo consumo en GPU VRAM

Contexto Medio (32.000 Tok)

Operaciones Matriz
1.024 millones de cálculos

Demanda de Memoria
Multiplicación por 64x

Contexto Largo (128.000 Tok)

Operaciones Matriz
16.384 millones de cálculos

Demanda de Memoria
Multiplicación por 1.024x

Fuente: Tay et al. (2022) Efficient Transformers: A Survey

Avances Clave: FlashAttention y Reutilización de KV Cache

Técnicas de hardware para superar la barrera cuadrática en servidores cloud

FlashAttention (I/O Aware)

Cómputo en Memoria SRAM Local
Evita escribir matrices gigantes en VRAM

Tiling de Bloques Pequeños
Fusión de operaciones matemáticas en GPU

Aceleración de 2x a 4x en Inferencia

Reutilización de KV Cache

Almacenamiento de Claves y Valores
No recalcula tokens pasados en cada paso

Consumo de Memoria VRAM Dinámico
Crece linealmente con cada nuevo token

Indispensable en vLLM y Bedrock

Fuente: Dao et al. (2022) FlashAttention: Fast and Memory-Efficient Exact Attention

Estructura de Precios: Tokens de Entrada frente a Salida

Por qué la generación de respuestas cuesta entre 3 y 5 veces más que el contexto de entrada

Tokens de Entrada (Prompt)

Procesamiento Paralelo Masivo
Fase Prefill: GPU satura núcleos de cálculo

Bajo Tiempo de Ocupación por Token
Miles de tokens procesados en milisegundos

Tarifa Base Reducida
Costo estándar por 1M de tokens

Tokens de Salida (Completion)

Generación Estrictamente Secuencial
Fase Decode: 1 token por ciclo de reloj

Retención Prolongada de Memoria VRAM
Monopoliza la GPU durante varios segundos

Tarifa Elevada (3x a 5x Más Cara)
Compensa la exclusividad del acelerador

Fuente: AWS Bedrock Pricing Model · FinOps Foundation Guide for GenAI

Tarifas Oficiales en Amazon Bedrock (Noviembre 2026)

Costos de referencia por 1M de tokens bajo demanda (US East N. Virginia)

Modelo Fundacional	Perfil de Carga	Entrada (1M Tok)	Salida (1M Tok)
Amazon Nova Micro	Texto rápido y clasificación	**$0.035 USD**	**$0.140 USD**
Amazon Nova Lite	Multimodal económico	**$0.060 USD**	**$0.240 USD**
Amazon Nova Pro	Propósito general avanzado	**$0.800 USD**	**$3.200 USD**
Claude 3.5 Haiku	Alta velocidad y precisión	**$0.800 USD**	**$4.000 USD**
Claude 3.5 Sonnet	Razonamiento complejo y código	**$3.000 USD**	**$15.000 USD**
Claude 3 Haiku (modelo histórico)	Generación anterior de referencia	$0.250 USD	$1.250 USD

Fuente: aws.amazon.com/bedrock/pricing/ (Consultado: Noviembre 2026)

Estimación de Costos FinOps: Metodología Práctica

Simulación para 50.000 peticiones mensuales · Ejemplo para el cálculo, no es tarifa real

Dimensionamiento Mensual (50.000 Consultas Empresariales)

Volumen Entrada por Consulta

1.000 tokens prompt = 50M tokens/mes

Volumen Salida por Consulta

300 tokens respuesta = 15M tokens/mes

Escenario A: Amazon Nova Pro ($0.80 / $3.20)

Entrada: 50 × $0.80 = $40.00 USD

Salida: 15 × $3.20 = $48.00 USD

Total Mensual: $88.00 USD

Escenario B: Claude 3.5 Sonnet ($3.00 / $15.00)

Entrada: 50 × $3.00 = $150.00 USD

Salida: 15 × $15.00 = $225.00 USD

Total Mensual: $375.00 USD

Conclusión FinOps: Enrutar a Nova Pro ahorra $287.00 USD mensuales (76 % de reducción)

Fuente: Metodología FinOps Cloud · Tarifas de referencia Amazon Bedrock (noviembre 2026)

Prompt Caching: Reutilización Inteligente de Contexto

Reducción del 90 % en tokens de entrada reutilizando prefijos estáticos en Bedrock

1. Primera Consulta (Cache Write)

Prefijo Estático (10.000 Tokens)
Manual técnico o reglamento corporativo

Procesamiento y Creación de KV Cache
Costo estándar de entrada (100 %)

Persistencia activa en memoria durante 5 minutos

2. Siguientes Consultas (Cache Read)

Reutilización Instantánea de KV Cache
No recalcula los 10.000 tokens base

Descuento del 90 % en Tarifa de Entrada
Claude 3.5 Sonnet: solo $0.30 / 1M tokens

Latencia TTFT reducida hasta un 80 %

Fuente: AWS Bedrock Prompt Caching Documentation · Anthropic Caching Guide

Cuatro Reglas de Oro para Gestionar Tokens

Directrices técnicas para optimizar rendimiento y costos en producción

1. Diseñar Prompts con Prefijo Estático

Ubicar instrucciones y manuales al inicio del prompt

Maximizar la tasa de acierto de Prompt Caching

Dejar las variables y preguntas al final de la cadena

2. Limitar el Parámetro max_tokens

Fijar un tope estricto al volumen de salida en la llamada API

Prevenir bucles infinitos de generación o verbosidad

Controlar el costo en la fase más cara de la inferencia

3. Auditoría de Tokens en Español

Medir el ratio real de tokens por palabra en producción

Seleccionar modelos con vocabularios de 128k+ tokens

Reducir el sobrecosto financiero regional en un 25 %

4. Truncamiento Inteligente de Contexto

No inundar el modelo con documentos RAG irrelevantes

Aplicar re-ranking para enviar solo fragmentos precisos

Mantener la latencia y la memoria en rangos óptimos

Fuente: AWS Generative AI Lens · Production Best Practices

HOJA DE RUTA

Hacia la Parte 3: El Pipeline de Inferencia en la Nube

De los costos y tokens a la ingeniería de servidores, motores vLLM y métricas de latencia

Contenidos de la Parte 3

1. Arquitectura de Inferencia Extremo a Extremo
API Gateways, balanceadores y servidores de GPU dedicados

2. Motores de Inferencia: Continuous Batching
Cómo vLLM optimiza el procesamiento concurrente en GPUs

3. Métricas Clave de Rendimiento: TTFT y TPOT
Medición de latencia de primer token y velocidad de generación

SEMANA 14 · INTELIGENCIA ARTIFICIAL EN LA NUBE I

IA en la Nube I: Modelos, Tokens e Inferencia

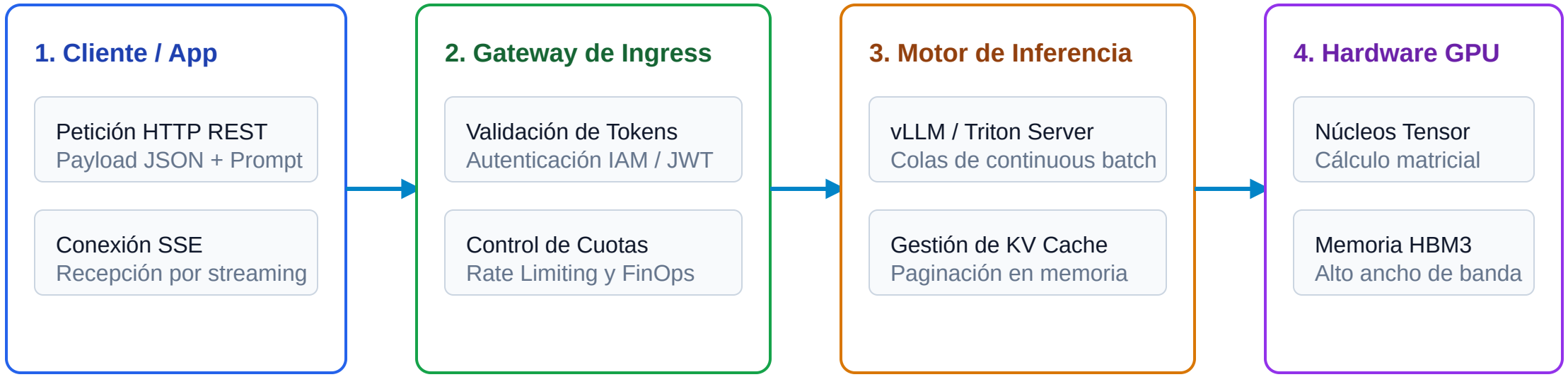
Parte 3 de 4 · Pipeline de Inferencia, Motores y Métricas de Rendimiento

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 14 · Periodo 2026-2 · Barranquilla, Colombia

Parte 3 de 4 · Pipeline de Inferencia, Motores y Métricas de Rendimiento

Flujo Integral de una Petición de Inferencia

Trayectoria completa desde la aplicación cliente hasta el acelerador de cómputo



Fuente: AWS Solutions Architecture · Generative AI Serving Reference Architecture

Capa Perimetral: Control de Acceso y Throttling

Defensas en el API Gateway para proteger la capacidad de cómputo y el presupuesto

1. Autenticación y RBAC

Validación de Tokens JWT
Firma criptográfica en Gateway

Permisos por Modelo
Control de acceso según rol

2. Rate Limiting (Throttling)

Algoritmo Token Bucket
Límites de peticiones por segundo

Protección Anti-Denegación
Evita saturación de la GPU

3. Gobernanza de Cuotas

Presupuestos por Equipo
Límite mensual de dólares o tokens

Respuesta HTTP 429
Degradación suave sin caídas

Fuente: AWS API Gateway Documentation · OWASP Top 10 for LLM Applications

Arquitectura Interna del Motor de Inferencia

Componentes clave que maximizan la utilización del acelerador en producción

1. Planificador (Scheduler)

Cola de Peticiones
Priorización First-Come First-Served

Continuous Batching
Agrupación a nivel de iteración

2. Gestor de Memoria (Block)

Tabla de Bloques Lógicos
Asignación dinámica de KV Cache

Eliminación de Fragmentación
Aprovecha el 96 % de la VRAM

3. Ejecutor del Modelo (Worker)

Kernel CUDA Optimizado
Ejecuta multiplicaciones en GPU

Tensor Parallelism
Distribución entre múltiples GPUs

Fuente: Kwon et al. (2023) Efficient Memory Management for Large Language Model Serving with PagedAttention

Continuous Batching: Eliminación de Tiempos Muertos

Agrupamiento dinámico a nivel de token en lugar de esperar a que termine la petición más lenta

Batching Estático Tradicional

Espera al Miembro Más Lento
El lote completo se bloquea hasta terminar

Desperdicio por Padding Masivo
GPU calcula multiplicaciones con ceros

Bajo rendimiento por dólar invertido

Continuous Batching (vLLM / Triton)

Planificación por Iteración de Token
Si una petición termina, libera su espacio

Inserción Inmediata de Nuevas Consultas
Nuevos prompts ingresan en el siguiente paso

Incremento de Throughput de 3x a 5x

Fuente: Yu et al. (2022) Orca: A Distributed Serving System for Transformer-Based Generative Models

Paginación de KV Cache: Memoria sin Fragmentación

Inspiración del sistema operativo: páginas virtuales no contiguas para almacenar contexto

Asignación Contigua Antigua

Reserva por Escenario Peor (Worst-Case)
Reserva memoria para el max_tokens completo

Severa Fragmentación Interna
Hasta el 70 % de la VRAM quedaba vacía

Límite muy bajo de usuarios concurrentes

Paginación Virtual de Memoria

Bloques Pequeños bajo Demanda
Páginas de 16 tokens en bloques físicos

Cero Desperdicio de VRAM
Los bloques físicos no necesitan ser contiguos

Aumenta concurrencia de 2x a 4x

Fuente: Kwon et al. (2023) PagedAttention · vLLM Project

Las Dos Fases en Acción: Prefill frente a Decode

Comportamiento del hardware durante la ingesta y la generación secuencial

Comportamiento Temporal de una Petición de Inferencia

Fase 1: Prefill

Procesa Prompt Completo

Alta ocupación de núcleos

Duración: 20 a 100 ms

Métrica: TTFT

Fase 2: Decode (Generación Secuencial)

Token 1 -> Token 2 -> Token 3 ... -> Token N

Limitado por lectura de pesos en memoria VRAM

Duración: 15 a 40 ms por token (acumula varios segundos)

Métrica: TPOT / TPS

Latencia Total = TTFT + (TPOT × Cantidad de Tokens de Salida)

Fuente: NVIDIA Technical Blog · LLM Inference Performance Analysis

Métricas Esenciales: TTFT, TPOT y Throughput

Parámetros técnicos para definir SLAs y dimensionar capacidad en la nube

TTFT (Primer Token)

Time to First Token
Desde envío hasta primer byte

Objetivo SLA
< 500 ms en interfaces interactivas

TPOT (Por Token Salida)

Time Per Output Token
Latencia individual entre tokens

Velocidad de Lectura
30 a 80 tokens/segundo (fluidez)

Throughput del Servidor

Tokens Totales / Segundo
Capacidad agregada multi-usuario

Métrica de Costo Cloud
Define cuántas GPUs aprovisionar

Fuente: AWS Bedrock Service Level Agreement · MLPerf Inference Benchmarks

Streaming con Server-Sent Events (SSE)

Entrega progresiva de tokens para reducir drásticamente la latencia percibida

Sin Streaming (Bloqueante)

Espera la Respuesta Completa
El cliente no recibe nada durante 5 a 10 seg

Pobre Experiencia de Usuario
Sensación de bloqueo o congelamiento

Riesgo de timeout en proxies HTTP

Con Streaming (Server-Sent Events)

Canal HTTP Abierto Unidireccional
Transmite cada token según se genera

Respuesta Visual en < 300 ms
El usuario lee mientras el modelo sigue generando

Estándar obligatorio en Amazon Bedrock

Fuente: MDN Web Docs: Server-sent events · AWS Bedrock InvokeModelWithResponseStream

Curva de Concurrencia y Saturación de la GPU

Cómo escala la latencia y el throughput según la cantidad de usuarios simultáneos

Evolución de Latencia y Throughput vs Concurrencia

Zona 1: Subutilizada

1 a 4 usuarios paralelos

Latencia mínima óptima

Costo por token muy elevado

Zona 2: Punto Dulce

8 a 32 usuarios paralelos

Throughput máximo GPU

Latencia dentro de SLA (< 1s)

Zona 3: Saturación

> 64 usuarios concurrentes

VRAM KV Cache al límite

Latencia se dispara exponencial

Directriz Cloud: Escalar horizontalmente antes de ingresar a la Zona 3

Fuente: MLPerf Inference Benchmarks · AWS Inferentia Capacity Guide

Hardware Cloud: GPUs NVIDIA frente a AWS Inferentia

Comparativa entre procesadores gráficos generales y chips ASIC especializados

GPUs NVIDIA (A10G, L4, A100, H100)

Ecosistema CUDA Universal
Compatibilidad absoluta con cualquier modelo

Máxima Flexibilidad y Rendimiento
Soporta tanto entrenamiento como inferencia

Costo por hora más elevado en la nube

AWS Inferentia2 (Instancias Inf2)

ASIC Especializado en Inferencia
Diseñado exclusivamente por Amazon para LLMs

Ahorro de Costo hasta un 50 %
Menor tarifa por hora que instancias GPU equivalentes

Requiere compilar con AWS Neuron SDK

Fuente: AWS EC2 Instance Types Guide · NVIDIA Tensor Core Technical Brief

Auto Scaling en Clústeres de Inferencia

Métricas reactivas y predictivas para escalar servidores GPU bajo demanda

1. Métricas de Activación

- Peticiones Concurrentes
- Uso de KV Cache VRAM (> 80 %)
- No usar CPU tradicional

2. Estrategia Warm Pool

- Instancias Pre-inicializadas
- Pesos precargados en memoria
- Arranque en < 30 segundos

3. Scale-Down Cauteloso

- Cooldown Prolongado
- Evitar efecto de oscilación
- Drenado limpio de tokens

Fuente: AWS Auto Scaling User Guide · Amazon ECS / EKS Best Practices

Observabilidad y Telemetría en Inferencia

Métricas, trazas y registros esenciales para garantizar SLA y control FinOps

1. Métricas Operativas

Latencias TTFT y TPOT
Monitoreo de percentil p95 y p99

Ocupación de la GPU
VRAM KV Cache y temperatura

2. Métricas de Consumo

Conteo de Tokens In / Out
Auditoría por cliente y modelo

Tasa de Acierto de Caché
Porcentaje de Prompt Caching

3. Auditoría y Trazas

Trazabilidad de Prompts
Rastreo de extremo a extremo

Alertas de Alucinación
Detección de patrones erráticos

Fuente: OpenTelemetry GenAI Semantic Conventions · Amazon CloudWatch

LECCIONES OPERATIVAS

Cuatro Fallas Clásicas en Inferencia y su Mitigación

Patrones de fallo frecuentes en centros de datos cloud y soluciones probadas

1. OOM (Out-of-Memory) en la GPU

Causa: Un usuario envía un prompt gigantesco no anticipado

Solución: Paginación en vLLM y rechazo estricto en el Gateway

Fijar max_model_len en la configuración del motor

2. Arranque en Frío Extremo (Cold Start)

Causa: Descarga de 20 GB de pesos desde S3 al autoescalar

Solución: Pools tibios (Warm Pool) e imágenes EBS precargadas

Reducción del tiempo de aprovisionamiento de 10 min a 30 seg

3. Timeouts en Proxies Intermedios

Causa: Peticiones largas sin streaming cerradas por el proxy

Solución: Habilitar streaming SSE y paquetes keep-alive

Configurar idle timeout del balanceador a más de 120 seg

4. Subutilización de GPU por Lote Pequeño

Causa: Despacho secuencial sin continuous batching

Solución: Motores modernos con planificación dinámica

Incrementa el aprovechamiento del hardware por encima del 80 %

Fuente: AWS Solutions Architecture · Field Experience & Incident Post-Mortems

Hacia la Parte 4: La Decisión de Despliegue

De los motores de inferencia a la estrategia de arquitectura: gestionado frente a autoalojado

Contenidos de la Parte 4

1. Los Tres Arquetipos de Despliegue
APIs Gestionadas (Bedrock) vs Servidor Cloud (EC2) vs Local (Ollama)

2. Soberanía del Dato y Marco Regulatorio en Colombia
Cumplimiento de la Ley 1581 de 2012 y residencia de datos confidenciales

3. Análisis de Punto de Equilibrio FinOps y LLM Routers
Cuándo conviene servidor propio y cómo enrutar dinámicamente

SEMANA 14 · INTELIGENCIA ARTIFICIAL EN LA NUBE I

IA en la Nube I: Modelos, Tokens e Inferencia

Parte 4 de 4 · Estrategia de Despliegue: Gestionado frente a Auto-Alojado

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 14 · Periodo 2026-2 · Barranquilla, Colombia

Parte 4 de 4 · Estrategia de Despliegue: Gestionado frente a Auto-Alojado

Los Tres Caminos Arquitectónicos para Desplegar IA

Delegación de infraestructura frente a soberanía, costo y control operativo

1. Gestionado (Bedrock)

Consumo Serverless
Invocación directa vía API

Facturación por Tokens
Cero costo fijo mensual

2. Servidor Cloud (EC2)

GPU Dedicada en AWS
Instancias G5 / Inf2 con vLLM

Costo Fijo Predecible
Tarifa horaria por instancia

3. Privado Local (Ollama)

Hardware On-Premises
Servidores propios en datacenter

Soberanía Total de Datos
Cero salida hacia Internet

Fuente: AWS Cloud Adoption Framework for Artificial Intelligence

Arquetipo 1: APIs Gestionadas con Amazon Bedrock

Inferencia serverless sin administración de servidores ni reservas de hardware

Arquitectura Gestionada en Amazon Bedrock

1. Acceso Unificado

API única para múltiples familias

Amazon Nova, Claude, Llama 3

Sin cambios de código base

Tiempo a producción: Horas

2. Seguridad Empresarial

Datos no se usan para re-entrenar

Cifrado KMS en reposo y tránsito

Conexión privada AWS PrivateLink

Aislamiento por cuenta VPC

3. Capacidades Avanzadas

Prompt Caching nativo (90 % off)

Batch Inference (50 % descuento)

Guardrails de contenido

Agentes y bases de conocimiento

EVALUACIÓN DE COMPROMISO

Evaluación del Enfoque Gestionado: Pros y Contras

Balance técnico entre agilidad de desarrollo y costo recurrente a escala

Ventajas Arquitectónicas

Cero Mantenimiento Operativo
Sin parches de kernel, CUDA ni fallas de GPU

Elasticidad Automática Infinita
Absorbe picos de demanda sin aprovisionar nodos

Modelos de Máxima Capacidad
Acceso a modelos propietarios de frontera

Desventajas y Riesgos

Costo Marginal a Gran Escala
Con millones de tokens diarios supera a una GPU propia

Dependencia Externa (Lock-in)
Sujeto a cuotas de tasa (TPM / RPM) del proveedor

Soberanía de Datos Foránea
Los datos viajan a regiones cloud en el extranjero

Fuente: Gartner Research: Cloud AI Platform Decision Matrix

Arquetipo 2: Servidor Cloud Dedicado con GPU en EC2

Instancias aceleradas gobernadas por contenedores y motores de inferencia vLLM

Arquitectura de Servidor Dedicado con GPU (EC2 g5.xlarge)

Capa de Aplicación: API REST FastAPI / OpenAI Compatible

Motor de Inferencia: Contenedor Docker vLLM con Continuous Batching

Pesos del Modelo: Meta Llama 3 8B Instruct (16 GB VRAM en FP16)

Hardware Subyacente: Instancia EC2 g5.xlarge (1 GPU NVIDIA A10G 24GB VRAM)

Costo Fijo Cloud: ~$1.00 USD / hora bajo demanda (~$730 USD / mes continuo)

Fuente: AWS Deep Learning AMIs Guide · vLLM Production Deployment Guide

EVALUACIÓN DE COMPROMISO

Evaluación del Servidor Cloud Dedicado: Pros y Contras

Control total del pipeline frente a costos fijos y complejidad de ingeniería MLOps

Ventajas Arquitectónicas

Costo Marginal Cero por Token
Económico para flujos intensivos 24/7

Control Absoluto del Motor
Ajuste de parámetros vLLM y cuantización

Latencia Altamente Consistente
Sin contención con otros inquilinos (No noisy neighbors)

Desventajas y Riesgos

Costo Fijo Ineludible
La factura corre igual si el servidor está inactivo

Carga Operativa de MLOps
El equipo debe gestionar parches, drivers y Auto Scaling

Límite Físico de Memoria
Modelos gigantes exigen clústeres multi-GPU costosos

Fuente: AWS EC2 Pricing Documentation · vLLM Production Benchmarks

Arquetipo 3: Inferencia Local y Privada con Ollama

Ejecución soberana en estaciones de trabajo y centros de datos corporativos

Inferencia Local con Ollama (Datacenter Propio / Workstation)

1. Simplicidad de Uso

Ejecución en 1 comando

'ollama run llama3'

Compatible con API OpenAI

Ideal para desarrollo local

2. Cuantización Automática

Formato binario GGUF

Cuantización a 4 u 8 bits

Ejecución en RAM y GPU mixta

Bajo consumo de memoria

3. Aislamiento Físico Total

Sin conexión a Internet

Datos médicos o financieros

Cero riesgo de filtración

Cumplimiento regulatorio estricto

Fuente: Ollama Project Documentation · Open-Source LLM Deployment Patterns

Soberanía del Dato y Marco Regulatorio en Colombia

Exigencias de la Ley 1581 de 2012 y normativas del sector financiero

1. Ley 1581 de 2012

Protección de Datos Personales
Consentimiento informado expreso

Transferencia Internacional
Restricción a países sin aval

2. Superfinanciera (Circ. 007)

Riesgo Operativo en Nube
Exigencia de auditoría y cifrado

Cláusulas de Privacidad
Custodia de llaves de cifrado propias

3. Solución Arquitectónica

Anonimización Previa
Enmascarar cédulas y nombres

Enrutamiento Híbrido
Datos privados en servidor local

Fuente: Ley 1581 de 2012 (Habeas Data) · Superintendencia Financiera de Colombia (Circular 007)

Punto de Equilibrio: ¿Cuándo Conviene Servidor Propio?

Metodología formal para calcular el cruce de costos entre Bedrock y EC2 dedicado

Fórmula del Punto de Equilibrio (Break-Even Point)

Punto de Equilibrio (Tokens/mes) = Costo Fijo Mensual Servidor ($730) / Costo Promedio por Token API

Frente a Modelo Avanzado ($3 / $15 por 1M)

Costo medio mixto: ~$6.00 USD / 1M tokens

Cruce en ~120 Millones de tokens / mes

Superado ese volumen, la GPU dedicada ahorra dinero

Frente a Modelo Ligero ($0.035 / $0.14 por 1M)

Costo medio mixto: ~$0.06 USD / 1M tokens

Cruce en ~12.000 Millones de tokens / mes

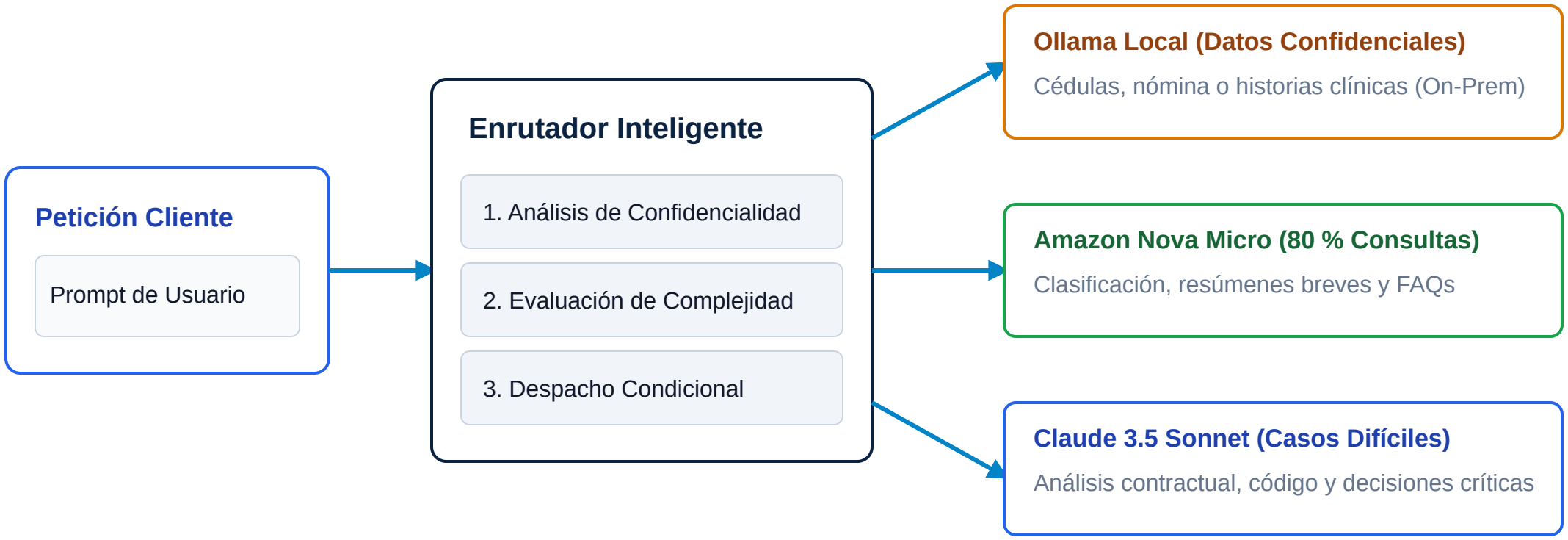
Más económico mantener API Serverless

Regla FinOps: Utilizar servidor dedicado solo cuando la utilización supere el 70 % continuo

Fuente: FinOps Foundation: Framework for AI Infrastructure · Ejemplo para el cálculo, no es tarifa real

Arquitectura Híbrida: El Enrutador Inteligente (LLM Router)

Enrutamiento dinámico según complejidad semántica, costo y confidencialidad



Fuente: AWS Architecture Center · Model Routing and Cascading Patterns

Topología Híbrida de Referencia para IA

Caso ilustrativo de arquitectura, no es una empresa real

Centro de Cómputo Local (On-Premises)

ERP y Base de Datos Transaccional
Información protegida y registros bancarios

Servidor Local Ollama (Llama 3 8B)
Procesamiento de datos confidenciales

Enrutador Perimetral con Túnel VPN IPsec

Nube Pública AWS (VPC Segura)

Amazon Bedrock (Nova Micro / Pro)
Inferencia elástica de alto volumen

Claude 3.5 Sonnet (Razonamiento)
Modelos de frontera con Prompt Caching

Conexión Privada AWS PrivateLink

Fuente: AWS Solutions Architecture · Hybrid Generative AI Enterprise Architecture

Seguridad en IA: Mitigación de Prompt Injection

Defensas por capas contra inyecciones directas, fugas de datos y alucinaciones

1. Separación de Contextos

Delimitar instrucciones del sistema con etiquetas XML

Impedir que el texto del usuario sobreescriba directrices

Principio de menor privilegio en tools y APIs

2. Guardrails Perimetrales

Filtros de contenido y detección de inyecciones

Enmascaramiento automático de datos PII (cédulas, tarjetas)

Bloqueo de respuestas sobre temas restringidos

3. Sanitización de Salidas

Validación estricta de esquemas JSON generados

Evitar inyección SQL indirecta desde el modelo

Verificación sintáctica antes de invocar bases de datos

4. Auditoría y Trazabilidad

Registro inmutable de prompts y respuestas en S3

Supervisión en tiempo real con CloudWatch

Detección de patrones maliciosos persistentes

Fuente: OWASP Top 10 for Large Language Model Applications · AWS Bedrock Guardrails

SÍNTESIS EJECUTIVA

Semana 14 en Cuatro Ideas Fundamentales

Principios directores para el arquitecto de nube en la era de la inteligencia artificial

1. Cambio de Paradigma Probabilístico

Los LLMs operan mediante matrices densas y probabilidades

La inferencia exige aceleración en GPUs y memoria HBM

La fase Decode está limitada por el ancho de banda de VRAM

2. El Token como Unidad de Medida y Costo

La tokenización en español consume entre 20 % y 40 % más

Los tokens de salida cuestan de 3x a 5x más que la entrada

Prompt Caching permite ahorrar hasta un 90 % en prefijos

3. Ingeniería de Motores de Inferencia

Continuous batching multiplica el throughput por 4x

La paginación virtual de KV Cache elimina fragmentación

TTFT y TPOT son las métricas soberanas de latencia

4. El Patrón Híbrido Triunfa en FinOps

Combinación de serverless en Bedrock y servidores dedicados

Enrutadores inteligentes para optimizar costo por consulta

Cumplimiento estricto de soberanía de datos en Colombia

HOJA DE RUTA

Adelanto de la Semana 15: GPU y Cuantización

De los conceptos de inferencia a la optimización profunda del hardware y los modelos

1. Arquitectura de GPUs

Núcleos Tensor y HBM3
Microarquitectura Hopper y Blackwell

Interconexión NVLink
Comunicación ultra-rápida multi-GPU

2. Cuantización de Modelos

Reducción de Precisión
De FP16 a INT8, INT4 y FP8

Técnicas AWQ y GPTQ
Compresión con mínima pérdida

3. Despliegue en Producción

Contenedores Especializados
Orquestación en Amazon EKS y K8s

Comparativa TCO Detallada
Modelos matemáticos a 3 años

Conclusión y Recursos de Estudio Recomendados

Documentación oficial y materiales de referencia para profundizar en IA cloud

Recursos de Estudio Recomendados

1. Documentación Oficial de Amazon Bedrock
Guías de arquitectura, Prompt Caching y parámetros de inferencia

2. Repositorio Oficial y Documentación de vLLM
Arquitectura de continuous batching y paginación de memoria

3. Material de Clase y Presentación en el Portal CUC
Diapositivas interactivas, guiones y lecturas complementarias